

Перед прочтением сжечь.

Положим, что по каналу связи осуществляется передача двоичных данных, впрочем, канал связи можно заменить устройством хранения данных. В результате воздействия шума, ошибок считывания информации, деградации носителя, единица ошибочно воспринимается, как ноль или ноль воспринимается, как единица. Рассмотрим упрощенную модель канала связи показанную на рисунке 1.

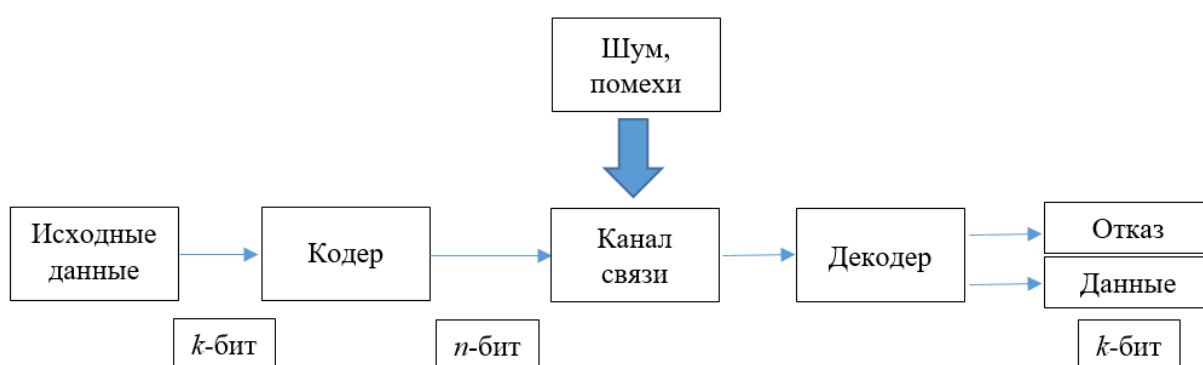


Рисунок 1. Модель канала связи

На рисунке 1 показана упрощенная модель системы передачи данных, но в рамках данного пособия такое упрощение допустимо. Примем, что данные передаются блоками фиксированной длины, также примем, что данные представлены в двоичном виде. Исходные данные в объеме k -бит попадают в так называемый кодер, где происходит их преобразование в вид пригодный для передачи или хранения. В результате работы кодера формируется блок из n бит, при этом, $n \geq k$. Фактически, можно сказать, что кодер по заранее известному алгоритму сопоставляет слова из двух множеств – множества слов длины k и множества слов длины n . Сформированные данные попадают в канал связи, тип которого в рамках настоящего пособия не имеет значения (это может быть устройство хранения данных). При работе канала связи в

передаваемые n могут быть добавлены искажения. Опишем процесс внесения искажений в передаваемый блок данных следующим выражением:

$$\tilde{F} = F + V,$$

где $\tilde{F}$ – блок данных на выходе канала связи, F – последовательность на входе канала связи, а V – вектор ошибки. Вектором ошибки будем называть двоичную комбинацию состоящую из n разрядов (по размеру передаваемого блока данных), в которой единицы расположены на тех позициях, где произошло искажение. При этом, в приведенном выражении суммирование ведется по модулю два, это вызвано тем, что было принято, что данные передаются в двоичном виде. В таблице 1 приведена таблица сложения по модулю два.

Таблица 1

$\oplus$	1	0
1	0	1
0	1	0

Приведем пример, пусть, осуществляется передача блока данных $F = 01011$, и в процессе передачи в старшем и младшем разрядах произошли искажения, тогда вектор ошибки будет равен $V = 10001$. В результате суммирования вектора ошибки и передаваемых данных будет получено $\tilde{F} = F + V = 01011 + 10001 = 11010$. Такая модель внесения ошибки в рамках решаемых задач будет достаточной. Сформированный в результате передачи по каналу связи блок $\tilde{F}$ попадает на вход декодера – устройства осуществляющего обратное кодирование преобразования, при этом, в ряде случаев декодер будет иметь возможность определения значения вектора ошибки V , и как следствие будет иметь возможность коррекции возникших искажений или формирования защитного отказа, в случае невозможности определения V . Разумеется, следует отметить, что при определенных способах работы кодера, декодер может вовсе не иметь возможности определения V .

Снизить влияние искажений возможно за счет аппаратных и алгоритмических решений. Аппаратные решения выходят за настоящее пособие и выполняемых в его рамках лабораторных работ. Алгоритмический способ – применение алгоритмов кодирования информации позволяющих преобразовать исходные данные таким образом, чтобы иметь возможность при декодировании определять значения вектора ошибки или формировать отказ о приеме блока данных.

Алгоритмы по которым работают кодер и декодер можно объединить в термин «код», иначе говоря код – это алгоритм прямого и обратного преобразования данных из одного вида в другой. Рассмотрим пример. Всем известна десятичная и двоичная система счисления. Обе эти системы счисления в полной мере могут считаться кодом – известны правила преобразования из одной системы счисления в другую и наоборот. Также следует отметить, что кодирование информации может осуществляться без цели контроля ошибок, а например, для удобства представления информации – в двоичном коде число по каналу связи передать существенно проще, чем в десятичном. В настоящем пособии будет рассмотрен только случай двоичных данных. Другими примерами можно считать ASCII – коды символов, почтовые индексы, номера маршрутов транспорта и др. Рассмотрим пример простого кода, который позволяет обнаружить ошибку. Пусть передается слово 110, $k = 3$, кодер осуществляет дублирование кодируемого слова, а результате $F = 110110$, $n = 6$. Декодеру известно, что первая половина слова $\tilde{F}$ поступившего из канала связи должна быть равна второй половине, таким образом декодирование сводится к их сравнению. В результате, если слово $\tilde{F}$ такую проверку не проходит, то формируется защитный отказ, в ином случае на выходе декодера формируется слово из $k = 3$ разрядов. Код с защитой повторением обладает двукратной избыточностью – на основе трех исходных разрядов формируется шесть. Рассмотрим другой пример – код с проверкой на четность. Кодер

осуществляет расчет количества единиц в слове, если оно нечетно, то в конце слова добавляется единица, в противном случае нуль:

$$100 \rightarrow 1001$$

$$101 \rightarrow 1010$$

Как видно из примера, $k = 3$, а $n = 4$. При выполнении декодирования осуществляется подсчет единиц, если оно нечетно, то в декодируемом слове возникла ошибка (нечетное число искаженных разрядов). Оба рассмотренных кода позволяют осуществлять контроль ошибок за счет внесения в исходные данные так называемой избыточности – дополнительной информации, которая не имеет для конечного пользователя никакого смысла, и используется только для обеспечения возможности контроля ошибок. Существует два способа описание уровня избыточности. Первый – это описание в виде так называемой скорости кода – отношения k/n , то есть отношения количества разрядов в слове на входе кодера к количеству разрядов в слове на его выходе. Вторым способом это определение величины избыточности:

$$D = \left(1 - \frac{\log_q M}{\log_q M_{\text{ни}}}\right) 100\%,$$

где D – величина избыточности в процентах, q – основание кода, фактически основанием кода является основание системы счисления (двойка, если код двоичный), M – мощность кода которая была бы достигнута в случае избыточного кода длины n , а $M_{\text{ни}}$ – мощность анализируемого, то есть избыточного кода (фактически полезных данных k разрядов, а передается n разрядов, $k \leq n$).

В рамках настоящего пособия будут рассмотрены различные подходы к алгоритмическому контролю ошибок.

1. Первичные коды

Первичными или простейшими кодами являются коды которые вовсе не обладают возможность контроля ошибок или только их обнаруживают¹, такие коды обычно используются для представления исходных данных, для формирования выходных величин датчиков, а также для представления данных на физическом уровне (в канале связи). Рассмотрим некоторые из них. Простейшим примером кода можно считать перевод из одной системы счисления в другую.

Двоичный нормальный код (двоичный равнодоступный код). Этот код имеет основание $q = 2$, а его мощность описывается выражением $M = q^n$, где n – длина слова. Пример - $7_{10} \rightarrow 111_2$

Троичный нормальный код. Этот код имеет основание $q = 3$, а его мощность также описывается выражением $M = q^n$, где n – длина слова. Пример - $8_{10} \rightarrow 22_3$

Оба рассмотренных кода не имеют возможности контроля ошибок.

Двоично-десятичный код. Данный код зачастую используется при реализации счетчиков, индикаторов, также следует отметить, что при его помощи можно представить дробные числа без потери точности. Этот код осуществляет преобразование каждого разряда десятичного числа в двоичный код. Основание этого кода $q = 2$, мощность описывается выражением $M = 10^{n/4}$. Рассмотрим пример $69_{10} \rightarrow 0110\ 1001_{2-10}$. Также следует отметить, что этот код может иметь несколько модификаций, в рассмотренном нами примере для кодирования каждого десятичного разряда был использован двоичный равнодоступный код длины 4. Вес каждого разряда двоичного равнодоступного кода равен степеням двойки, то есть 8-4-2-1, таким образом младший разряд тетрады кодирует единицу, а старший восьмерку. Пусть имеем число в двоичном равнодоступном коде равное 1011, его перевод в

¹ Это несколько спорное выражение, код обнаруживающий ошибки, как будет понятно далее, не всегда будет «простейшим».

десятичную форму записывается как сумма весов разрядов умноженных на значения самих разрядов: $1011_2 = 1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 8 + 2 + 1 = 11$. Следует отметить, что при помощи четырех двоичных разрядов кодируется одно десятичное число в диапазоне от нуля до девяти, таким образом, каждая тетрада будет иметь десять разрешенных значений, и шесть запрещенных, которые можно использовать для кодирования дополнительной информации, например запятой или кодирования команд. При этом существуют иные модификации двоичного кода, например 2-4-2-1, 6-3-2-1, это двоичные коды уже не являются равнодоступными.

Код с постоянным весом. Эти коды относятся к так называемым неразделенным или невзвешенным кодам – вес каждого слова кода строго фиксирован (количество и значения ненулевых разрядов в каждом слове). Любое слово этого кода всегда содержит строго φ символов одного вида. Пусть $\varphi_0 = 2$, $\varphi_1 = 3$, $\varphi_2 = 1$, а $n = 6$. Примерами слов принадлежащих этому коду могут являться следующие комбинации 010112, 121001 – эти слова содержат строго два нуля, три единицы и одну двойку, общая длина слова $n = \varphi_0 + \varphi_1 + \dots + \varphi_{q-1}$, при этом мощность этого кода определяется выражением:

$$M = \frac{n!}{\varphi_0! \varphi_1 \dots \varphi_{q-1}}$$

Частным случаем, при $q = 2$ является код на одно сочетание, мощность кода равна $M = C_n^m$, где m – число единиц в каждой комбинации принадлежащей коду. То есть каждая комбинация принадлежащая коду будет иметь равное количество единиц, например, код на два сочетания, при $n = 4$ будет иметь следующие комбинации: 1001, 0011, 0110, 1100, 1010, 0101. Как видно из приведенного примера, каждое слово имеет строго по две единицы, мощность этого кода определяется, как число сочетаний из n по m и равна $M = C_4^2 = 6$. Нетрудно догадаться, что такой код обладает возможностью контроля ошибок – декодер осуществляет подсчет веса каждого слова, контроль

значений разрядов, и на основании этого определяется наличие ошибки. Кодер для такого кода реализуется в виде таблицы соответствия исходных данных и слов кода с постоянным весом (об это будет подробнее сказано в последующих разделах настоящего пособия).

Код с проверкой на четность. Код с проверкой на четность (нечетность) был рассмотрен нами ранее, несколько формализуем существующие знания. Код с проверкой на четность формируется путем добавления нуля или единицы к кодируемой последовательности, в так чтобы общее число единиц в слове было четным (нечетным). Основание этого кода $q = 2$, а мощность $M = 2^{n-1}$. Пример кода с проверкой на четность: $7_{10} \rightarrow 111_2 \rightarrow 111(1)_{\text{КПЧ}}$, и пример с проверкой на нечетность $7_{10} \rightarrow 111_2 \rightarrow 111(0)_{\text{КПНЧ}}$. В скобках отмечен дополнительный разряд проверки четности (нечетности).

Манчестерский код (также можно встретить следующие названия: код с удвоением элементов, код Манчестер-II, корреляционный код). Кодирование сообщения осуществляется следующим образом. Сначала записывается очередной разряд кодируемого слова, а за ним записывается его инверсное значение, мощность этого кода равна $M = q^{n/2}$. Рассмотрим пример: $110_2 \rightarrow 10\ 10\ 01_{\text{МII}}$, таким образом, каждый разряд исходного слова записывается в виде двух разрядов – его исходного и инверсного значений, в примере для удобства разряды записаны через пробел.

Код с защитой повторением. В данном случае кодируемое сообщение записывается несколько раз подряд, $M = q^{n/r}$, где r – количество повторений. Например, пусть $n = 6$, $r = 3$: $01_2 \rightarrow 01\ 01\ 01_{\text{КЗП}}$. При декодировании известно количество повторений и общая длина слова, таким образом декодер осуществляет деление слова на фрагменты и их сравнение.

Код с защитой повторением и инверсией. В данном случае кодируемое сообщение записывается дважды, но если число единиц в кодируемом слове четное, то повторение происходит без изменений, в противном случае при

повторении разряды заменяются на инверсные, $M = q^{n/2}$. Например, пусть $n = 4$: $01_2 \rightarrow 01\ 10_{\text{кзпи}}$, $11_2 \rightarrow 11\ 11_{\text{кзпи}}$

В последующих разделах настоящего пособия будет рассмотрено, как избыточность влияет на количество ошибок с которыми сможет работать код.

2. Кодовое расстояние

Ранее было сказано, что одним из способов защиты информации от искажений является преобразование исходных данных. Рассмотрим теоретическую основу такого преобразования. Очевидно, что одна комбинация может быть преобразована в другую путем изменения хотя бы одного разряда, «половину» разряда изменить невозможно. Количество разрядов, на которое пара комбинаций отличается друг от друга называется кодовым расстоянием или расстоянием Хэмминга и обозначается строчной буквой d . Пусть имеется две комбинации $\{1,1\}$ и $\{1,0\}$, если осуществить поразрядное сравнение этих комбинаций, то очевидно, что они отличаются между собой в одном разряде, соответственно кодовое расстояние между этой парой комбинаций равно единице. Рассмотрим иную пару комбинаций: $\{1,1\}$ и $\{0,0\}$, очевидно, что кодовое расстояние для такой пары равно двум.

Теперь введем следующую абстракцию. Набор комбинаций длины n образует n – мерное пространство, в котором каждая комбинация является центром сферы единичного радиуса, в пределах этой сферы комбинация остается неизменной («половину» разряда изменить нельзя, как было отмечено выше). В зависимости от количества комбинаций в таком пространстве сферы могут находиться на удалении друг от друга, а также могут пересекаться. Рассмотрим следующую пару комбинаций $A = \{1,1\}$ и $B = \{1,0\}$, кодовое расстояние между которыми равно единице. При этом, будем считать, что обе комбинации несут полезную информацию и в этом смысле являются равноправными, такие комбинации будем называть разрешенными, и будем считать, что в них ошибок нет. Данные комбинации имеют длину равную двум, и как следствие, находятся в двумерном пространстве. В этом случае сферы центрами которых

являются данные комбинации становятся окружностями, как приведено на рисунке 2.

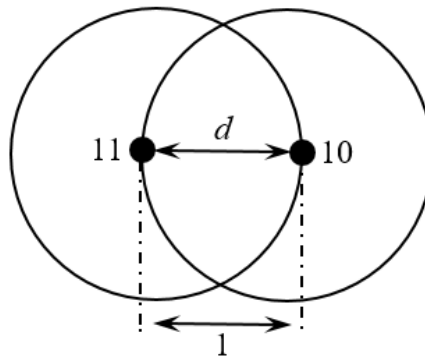


Рисунок 2. Окружности образуемые комбинациями $\{1,1\}$ и $\{1,0\}$

Как видно из рисунка 2, границы окружностей пересекаются, центр окружности комбинации А находится на границе окружности комбинации В, и наоборот, соответственно. Пересечение окружностей вызвано тем, что кодовое расстояние между этими комбинациями равно единице. В случае, если в результате помех и искажений в комбинации $A = \{0,1\}$ будет искажен старший разряд, то произойдет переход в окружность комбинации В. Таким образом, отследить такое искажение невозможно – произошел переход от одной разрешенной комбинации к другой. Теперь рассмотрим несколько иной случай. Пусть имеются две разрешенные комбинации $A = \{0,0\}$ и $B = \{1,1\}$, кодовое расстояние между которыми равно двум, окружности образуемые этой парой комбинаций показано на рисунке 3.

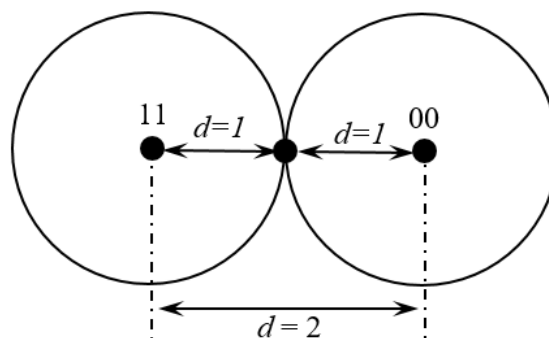


Рисунок 3. Окружности образуемые комбинациями $\{0,0\}$ и $\{1,1\}$

Как видно из рисунка 2, увеличение кодового расстояния между разрешенными комбинациями приводит к тому, что окружности имеют только одну точку пересечения, а не область, как в случае комбинаций $\{1,1\}$ и $\{1,0\}$. Переход от одной разрешенной комбинации к другой происходит изменением двух разрядов и возможен только через комбинацию «01» или «10». Таким образом, если принять комбинации «01» или «10» запрещенными, то становится возможным определить наличие однократной ошибки, которая может возникнуть в комбинациях $\{1,1\}$ и $\{0,0\}$.

Дальнейшее увеличение кодового расстояния приводит к увеличению длины слова и увеличению количества запрещенных комбинаций, рассмотрим следующий пример. Пусть имеется две разрешенные комбинации $\{0,0,0\}$ и $\{1,1,1\}$, кодовое расстояние между этими комбинациями равно трем, теперь рассмотрим единичные окружности образуемые этими комбинациями, как показано на рисунке 4.

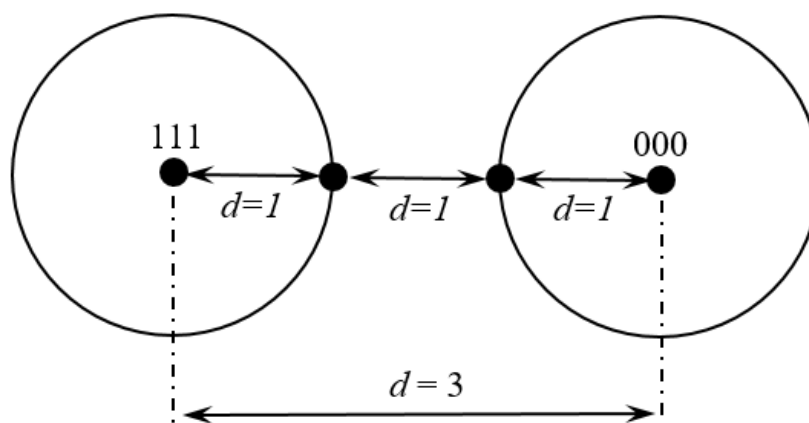


Рисунок 4. Окружности образуемые комбинациями $\{0,0,0\}$ и $\{1,1,1\}$

Как видно из рисунка 4, переход от комбинации $\{0,0,0\}$ к $\{1,1,1\}$ происходит не через одну запрещенную комбинацию, а через две $\{0,0,1\}$ и $\{0,1,1\}$. При этом, кодовое расстояние между разрешенными и запрещенными комбинациями неравно – запрещенная комбинация $\{0,0,1\}$ «ближе» к разрешенной комбинации $\{0,0,0\}$, а комбинация $\{0,1,1\}$ ближе к $\{1,1,1\}$. В

этом случае становится возможным не только определить наличие ошибки, но и выполнить ее коррекцию. В случае, если из канала связи или иного носителя информации была получена запрещенная комбинация, то возможно определить ближайшую в смысле кодового расстояния разрешенную комбинацию, тем самым исправив ошибку. Дальнейшее увеличение кодового расстояния приводит к росту количества обнаруживаемых и исправляемых ошибок. Теперь рассмотрим случай увеличения кодового расстояния до величины равной пяти, случай $d = 4$ читателю предлагается иллюстрировать самостоятельно. На рисунке 5 показан такой случай.

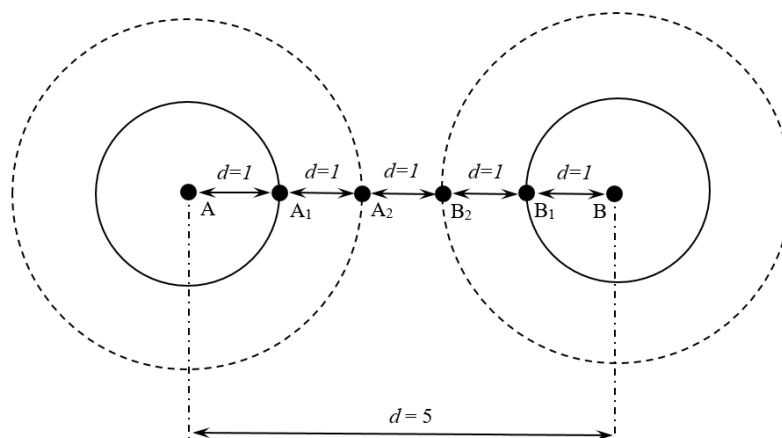


Рисунок 5. Случай кодового расстояния равного пяти

На рисунке 5 приведены две разрешенные комбинации A и B, между которыми кодовое расстояние равно пяти. Кроме этого, рисунок содержит два типа окружностей – малого и большого радиусов, которые изображены сплошной и пунктирной линией, соответственно. Центрами каждой пары окружностей является соответствующая разрешенная комбинация. Анализируя рисунок, нетрудно догадаться, что комбинации A_1 и A_2 декодируются в комбинацию A, а комбинации B_1 и B_2 декодируются в B. Таким образом, при кодовом расстоянии равном пяти становится возможным исправить двукратную ошибку. Теперь установим более формально взаимосвязь между числом исправляемых ошибок, которое будем обозначать строчной буквой t и

кодовым расстоянием. Как видно из рисунка, окружность изображенная пунктиром включает в себя комбинацию A_1 , A_2 и A , такую окружность принято называть сферой декодирования комбинации, которая находится в ее центре. Таким образом, окружность в левой части рисунка является сферой декодирования комбинации A , аналогичны будут рассуждения и для комбинации B . Как видно из рисунка, радиус сферы декодирования равен количеству исправляемых ошибок. При этом, «зазор» между соседними сферами декодирования равен единице, иначе говоря, «зазор» между сферами декодирования позволяет обеспечить принадлежность комбинаций находящихся в пределах сфер декодирования к их центрам. Если данное условие не выполнить, и предположить, что кодовое расстояние между A_2 и B_2 равно единице, то эти комбинации будут равноудалены от A и B . Отсюда можно сделать вывод, что для обеспечения перехода от одной разрешенной комбинации к сфере декодирования ближайшей разрешенной кодовой комбинации потребуется сначала выйти на границу сферы декодирования, то есть выполнить $2t$ переходов (по числу исправляемых ошибок и радиусу сферы декодирования), комбинации находящиеся на расстоянии не более, чем $2t$ от разрешенной, еще могут быть корректно декодированы. Таким образом, кодовое расстояние с количеством исправляемых ошибок будет связано выражением $d \geq 2t + 1$, где t – число исправляемых ошибок. Из этого выражения следует, что рост кодового расстояния приводит к увеличению радиуса сфер декодирования каждой разрешенной комбинации, и как следствие к росту количества запрещенных комбинаций.

Введем определение величины кодового расстояния кода. В приведенных примерах был рассмотрен код состоящий только из двух разрешенных комбинаций, в результате чего кодовое расстояние определялось только между одной парой комбинаций. Разумеется, что на практике, код будет содержать существенно большее количество разрешенных комбинаций, и в этом случае кодовое расстояние определяется, как минимальное кодовое

расстояние между всеми парами разрешенных комбинаций. Забегая вперед, справедливо отметить, что кодовое расстояние можно определить не только перебором всех пар комбинаций, а найдя комбинацию с минимальным весом, который и будет равен кодовому расстоянию. Вес комбинации определяется, как число ненулевых ее компонент, например, комбинация $\{101\}$ имеет вес равный двум, а комбинация $\{0111\}$ равный трем. Однако, такой подход разумно применять только в том случае, если код является линейным и если известно, что в исследуемом множестве разрешенных комбинаций нет повторов. Множество разрешенных комбинаций линейного кода характеризуется тем, что сумма двух комбинаций также является разрешенной комбинацией этого кода.

В последующих разделах мы будем неоднократно обращаться к понятию сферы декодирования и вопросам их построения, целью данного раздела являлось понимание взаимосвязи величины кодового расстояния и числа исправляемых ошибок. Подведем небольшой итог. Для обеспечения контроля ошибок, набор комбинаций приходится делить на два множества – разрешенных и запрещенных комбинаций, при этом между всеми парами разрешенных комбинаций должно поддерживаться кодовое расстояние не менее $2t+1$. Каждая разрешенная комбинация является центром сферы декодирования, которая включает в себя запрещенные комбинации, которые отстоят в смысле кодового расстояния от центра сферы на величину не более $2t$. Запрещенные комбинации находящиеся в пределах сферы декодирования, декодируются в разрешенную комбинацию, которая является центром соответствующей сферы. Отсюда очевидным становится понятие величины избыточности о которой было сказано ранее – фактически часть комбинаций перестают участвовать в представлении данных и становятся «ошибочными». Приведем (в очередной раз) пример кода который исправляет одну ошибку. Пусть длина двоичной комбинации равна трем, то есть мощность на этой длине $M = 2^3 = 8$, то есть такая длина позволяет формировать восемь комбинаций:

$\{000, 001, 010, 011, 100, 101, 110, 111\}$. При этом, для обеспечения кодового расстояния будут использованы две разрешенные комбинации – $\{000, 111\}$. Остальные комбинации становятся запрещенными, комбинации $\{001, 010, 100\}$ будут декодированы в 000. Справедливо будет сказать, что комбинация 001 это комбинация 000, но искаженная в младшем разряде, 010 во втором, а 100 в старшем. Комбинации $\{011, 110, 101\}$ декодируются в 111. В дальнейшем мы будем неоднократно обращаться к данному набору комбинаций для пояснений рассматриваемого материала.

3. О простейшем алгоритме построения кода с контролем ошибок

Каждая разрешенная кодовая комбинация и отстоящие (в смысле кодового расстояния) от нее запрещенные комбинации на величину не более, чем количество исправляемых ошибок образуют, как было сказано ранее, так называемую сферу декодирования, центром которой является разрешенная комбинация. Для каждой запрещенной комбинации находящейся внутри сферы декодирования возможно однозначно определить соответствующую ей разрешенную комбинацию. Такие запрещенные комбинации иногда в литературе называют комбинациями-спутниками. В случае четной величины кодового расстояния формируются запрещенные комбинации, которые находятся на границах сфер декодирования, в этом случае такие ошибки только обнаруживаются. Таким примером является уже рассмотренный выше случай, при котором кодовое расстояние равно двум.

3.1. Простейший способ построения кода обнаруживающего одну ошибку

Рассмотрим построение кода позволяющего вести контроль ошибок с учетом введенного ранее определения величины кодового расстояния. Но перед тем, как осуществить построение кода с возможностью контроля ошибок, определим множество исходных (кодируемых) комбинаций. Пусть требуется осуществить представление M слов в двоичном виде, для этого нужно

определить количество разрядов достаточное для формирования нужного количества слов. Величина M называется мощностью кода, и связана с длиной слова принадлежащего коду следующим выражением:

$$M \leq q^k,$$

где q – количество символов в алфавите кода (грубо говоря, основание системы счисления), k – количество разрядов, в двоичном случае q равно двум. Таким образом, при известной требуемой мощности становится возможным определить требуемую длину слова. При этом, выбрав величину k минимально возможной для выполнения приведенного выше неравенства, становится возможным построить без избыточных комбинаций или с минимально возможным их количеством. Приведем два примера – необходимо обеспечить мощность равную двум и равную трем. В первом случае достаточно одного двоичного разряда, то есть, $k = 1$, таким образом, будут использованы обе возможные комбинации. Во втором случае, потребуется $k = 2$, что позволяет формировать уже четыре комбинации, таким образом, две комбинации оказывается избыточной. Независимо от наличия избыточных комбинаций, такой код будет иметь кодовое расстояние равное единице и его можно считать кодом без избытка. Теперь перейдем к построению кода имеющего возможность контроля ошибок. Пусть требуется осуществить двоичное представление следующего набора данных $\{0,1,2,3\}$ с возможностью обнаружения одной ошибки, то есть кодовое расстояние такого кода должно быть равно двум. Нам требуется обеспечить кодирование четырех слов, то есть мощность кода равна четырем. Обеспечить такую мощность позволяет $k = 2$. В результате чего формируется двоичный код ζ_2 без избытка, который можно представить в виде таблицы:

Таблица 2. Соответствие исходной информации и кода без избытка

Исходные данные	Комбинации кода ζ_2
0	00
1	01
2	10
3	11

Используя такую таблицу возможно, как при кодировании, так и при декодировании, таким образом однозначно можно установить соответствие между комбинациями кода ζ_2 и исходными данными. Однако, код ζ_2 обладает кодовым расстоянием равным единице, что не позволит вести работу с ошибками, в этом просто убедиться, если выполнить попарное определение кодового расстояния между комбинациями кода ζ_2 . Как было сказано выше, необходимо добиться кодового расстояния равного двум (обеспечение выявления одной ошибки). Для этого пойдем по пути увеличения длины кодового слова, примем длину кода $n = 3$, такая длина позволяет достичь мощности равной восьми, но при этом, нами будет использовано по прежнему только четыре комбинации, таким образом, становится возможным сформировать набор разрешенных и запрещенных комбинаций. При этом, для набора разрешенных комбинаций необходимо обеспечить кодовое расстояние равное двум. Построение кода начнем с формирования полного набора комбинаций β при длине $n = 3$: $\beta = \{000, 001, 010, 011, 100, 101, 110, 111\}$. Кодовое расстояние такого набора равно единице. Напомним, что для поставленной задачи нужно сформировать код ζ_4 с мощностью равной четырем, таким образом, количество разрешенных комбинаций будет равно четырем. После чего начнем последовательно формировать наборы α_i , которые получены путем выбора из набора β четырех комбинации, всего таких наборов может быть $C_8^4 = 70$. Такой процесс будем продолжать до тех пор, пока кодовое расстояния очередного α_i не будет равно требуемому или пока не будет

осуществлен перебор всех возможных α_i . На рисунке 6 проиллюстрирован такой процесс.

β				
000		α_1	α_2	α_j
001		000	000	000
010				
011	→	001	001	011
100		010	010	101
101		011	100	110
110				
111		$d = 1$	$d = 1$	$d = 2$

Рисунок 6. Процесс поиска набора с нужным кодовым расстоянием

На рисунке 6 в первом столбце изображен набор всех комбинаций β получаемых на длине равной трем, далее схематично показан процесс отбора кандидатов α_i в набор разрешенных комбинаций. Как видно, такой кандидат определен и на рисунке отмечен как α_j . Теперь нетрудно определить набор запрещенных комбинаций кода ζ_4 . Ниже приведены все комбинации принадлежащие этому коду.

Таблица 3. Комбинации кода ζ_4

Разрешенные комбинации кода ζ_4	Запрещенные комбинации кода ζ_4
000	001
011	010
101	100
110	111

После того, как нам стали известны разрешенные комбинации можно приступать к построению кодера, задача которого сводится к определению соответствия слов из кода ζ_2 словам из кода ζ_4 . По причине того, что разрешенные комбинации формировались перебором, без учета кодируемых комбинаций (принадлежащих коду ζ_2), то кодер формируется в виде таблицы задающий соответствия между словами этих двух кодов:

Таблица 4. Реализация кодера для кода ζ_4

Комбинации кода ζ_2	Разрешенные комбинации кода ζ_4
00	000
01	011
10	101
11	110

Такую таблицу удобно реализовать в виде массива соответствующего размера, который будет хранить комбинации кода ζ_4 , а комбинации кода ζ_2 будут играть роль индекса. Таким образом, операция кодирования будет сведена к получению комбинации по индексу. Разумеется, в случае, если исходные данные не являются двоичными числами, а например строками или иными данными, то понадобится дополнительная таблица по типу таблицы 2. Следует отметить, что в случаях более сложных кодов, чем построенный код ζ_4 способ кодирования показанный в таблице 4 применяется на практике, его достоинством является простота реализации и константное время кодирования, а недостатком является высокий расход памяти, при росте мощности кода.

Теперь перейдем к построению декодера для кода ζ_4 . Декодер будет осуществлять обратное преобразование из ζ_4 в ζ_2 , а в случае декодирования запрещенной комбинации сообщать об ошибке. Декодер также можно

реализовать в виде таблицы (массива), но размер такой таблице будет равен 2^n , то есть равно количеству разрешенных и запрещенных комбинаций кода ζ_4 . При этом, декодируемая комбинация будет выступать в роли индекса, при этом, в случае разрешенной комбинации следует хранить соответствующую комбинацию из ζ_2 , а в случае запрещенной комбинацию-маркер, например, комбинацию $\{111\}$. Пример таблицы-декодера показан в таблице 5.

Таблица 5. Реализация декодера для кода ζ_4

Декодируемые комбинации	Комбинации кода ζ_2 или комбинации-маркеры
000	00
011	01
101	10
110	11
001	111
010	111
100	111
111	111

Первые четыре строки таблицы 5 отвечают за разрешенные комбинации и им соответствуют комбинации кода ζ_2 , а последующие четыре строки за запрещенные комбинации и им соответствует комбинация $\{111\}$. Такую таблицу также, как и в случае кодера просто сделать при помощи массива.

3.2. Простейший способ построения кода исправляющего одну ошибку

Теперь перейдем к рассмотрению случая с возможностью коррекции однократной ошибки, примем следующие параметры кода ζ : мощность равна двум ($M = 2$), кодовое расстояние и длина кода равны трем ($d = 3, n = 3$). Выполнив процедуры аналогичные построению кода ζ_4 получим следующую разрешенные и запрещенные комбинации приведенные в таблице 6.

Таблица 6. Комбинации кода ζ

Разрешенные комбинации кода ζ	Запрещенные комбинации кода ζ
000	001
	010
	100
111	011
	101
	110

Как видно из таблицы 6, код ζ обладает большей избыточностью, чем код ζ_4 . Избыточность кода можно определить, например, как соотношение предельно достигаемой мощности при длине n к мощности используемого кода, таким образом, данный код обладает четырехкратным избытком. Кодер этого кода будет аналогичен кодеру кода ζ_4 который был рассмотрен ранее. Перейдем к построению декодера, для этого нужно более детально рассмотреть процесс определения запрещенных комбинаций. Для этого вернемся к понятию сфер или областей декодирования о котором было сказано ранее. Напомним, что у кода с возможностью коррекции ошибки, каждая разрешенная комбинация находится внутри так называемой сферы декодирования. Каждая сфера содержит помимо разрешенной комбинации набор запрещенных комбинаций, которые отстоят в смысле кодового расстояния от разрешенной комбинации на величину меньшую или равную количеству исправляемых кодом ошибок. Проиллюстрируем такие сферы для построенного кода ζ при помощи рисунка 7.

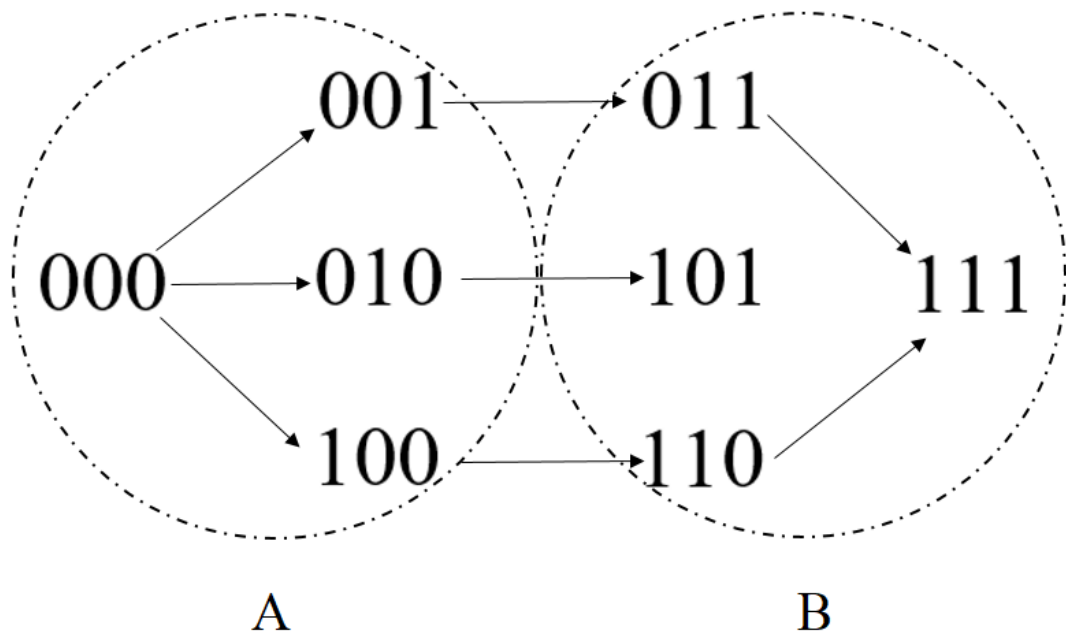


Рисунок 7. Сферы декодирования для кода ζ . А – область декодирования комбинации {000}, В – область декодирования комбинации {111}

Как видно из рисунка 7 для кода ζ образуется две сферы (области) декодирования², таким образом, при декодировании любой из комбинаций из области А можно считать, что это комбинация {000}, а для области В комбинация {111}. Напомним, что общее количество комбинаций при длине слова n равно величине 2^n , M комбинаций из этого набора являются разрешенными. Имея набор M разрешенных комбинаций нужно найти подходящие к ним запрещенные комбинации из $(2^n - M)$ комбинаций, для этого нужно осуществить перебор и найти для каждой разрешенной комбинации, набор комбинаций отстоящих на величину не более, чем t . Имея набор запрещенных комбинаций можно построить таблицу-декодер для кода ζ , которая приведена ниже.

Таблица 7. Реализация декодера для кода ζ

² В данном случае разрешенные комбинации для упрощения визуализации не являются центрами сфер декодирования.

Декодируемые комбинации	Разрешенные комбинации кода ζ
000	000
001	
010	
100	
111	111
011	
101	
110	

Также декодер можно построить без дополнительного формирования запрещенных комбинаций. Этот способ заключается в поиске наиболее близкой в смысле кодового расстояния разрешенной комбинации (при этом, величина на которую отстоит декодируемая комбинация от разрешенной комбинации не должна превышать величину t , иначе возможно ошибочное декодирование). Рассмотрим пример, при декодировании комбинации $\{010\}$ наиболее близкая к ней разрешенная комбинация оказывается $\{000\}$. Недостатком такого способа является различное время декодирования, при наличии или отсутствии ошибки.

3.3. Определение общей длины кода

У внимательного читателя может возникнуть вопрос определения длины кода, при известной требуемой мощности и известном кодовом расстоянии. Разумеется, что такую задачу можно решить перебором, при котором будет осуществляться последовательное увеличение длины кода и осуществляется попытка найти множество разрешенных комбинаций, которое бы удовлетворяло требуемой мощности и кодовому расстоянию. Такой способ весьма трудоемкий, есть иной путь решения – это использование оценочных

формул, которые связывают мощность, длину кода и кодовое расстояние. Ранее нами были рассмотрены сферы декодирования, на их основе выведем оценочную формулу, на основе которой можно определить предельно возможные параметры кода – мощность M , длину кода n , и кодовое расстояние d . В литературе обычно параметры кода записываются в виде выражения в скобках вида (n, k, d) или (n, k) . Каждая сфера декодирования содержит число комбинаций равное $\sum_{i=0}^t C_n^i$, а всего сфер декодирования будет равно мощности кода M . Поясним данное выражение на примере построенного кода ζ с кодовым расстоянием равным трем, которое позволяет исправить одну ошибку. Данная сумма будет иметь два элемента - C_n^0 и C_n^1 . Поясним подробнее. Сфера декодирования содержит в себе саму разрешенную комбинацию – это первый член суммы, который равен C_n^0 и равен единице. Каждая запрещенная комбинация отстоит от разрешенной на величину не более, чем t . Пусть $\{000\}$ – разрешенная комбинация, тогда, при $t = 1$ возможны три варианта искажений – $\{001, 010, 100\}$, число таких искажений также определяется, как число сочетаний из n по t . В нашем примере $t = 1$, отсюда $C_3^1 = 3$. Если ошибок больше, то и количество элементов в сумме будет больше. Как было сказано ранее, всего M сфер декодирования, тогда общее число комбинаций принадлежащих коду (разрешенных и запрещенных) будет равно произведению $M \sum_{i=0}^t C_n^i$. При этом, общее число комбинаций достигаемых на длине n равно 2^n , в это число входят разрешенные и запрещенные комбинации кода, а также комбинации которые отстоят от всех разрешенных комбинаций на величину больше, чем t . Отсюда справедливо следующее неравенство:

$$2^n \geq M \sum_{i=0}^t C_n^i$$

Данное выражение интерпретируется следующим образом – количество комбинаций длины n , входящих в сферы декодирования, при заданном t , не

должно превышать предельно возможное число комбинаций достигаемое на длине n . Используя это неравенство можно вывести верхнюю оценку границы мощности кода, которая может быть достигнута при заданных n и t :

$$M \leq \frac{2^n}{\sum_{i=0}^t C_n^i}$$

Данное выражение называется оценкой Хэмминга, а коды удовлетворяющие строгому неравенству называются оптимальными кодами или кодами с плотной упаковкой – в таких кодах все комбинации получаемые при длине n строго делятся на запрещенные и разрешенные или иначе говоря, все комбинации входят в ту или иную сферу декодирования.

3.4. Дополнительная информация о декодировании

Ранее нами некоторый код был рассмотрен, как набор комбинаций, часть из которых были приняты разрешенными, а часть запрещенными. При этом на разрешенные комбинации было наложено ограничение по кодовому расстоянию. В результате действия данного ограничения стало возможным формирование сфер декодирования разрешенных комбинаций. Было отмечено, что комбинации находящиеся внутри сфер декодируются в комбинацию находящуюся в центре сферы.

Но в случае, если код не оптимален, то между сферами декодирования формируется «пространство» заполненное запрещенными комбинациями, которые находятся на равном расстоянии от двух или более сфер, как схематично показано на рисунке

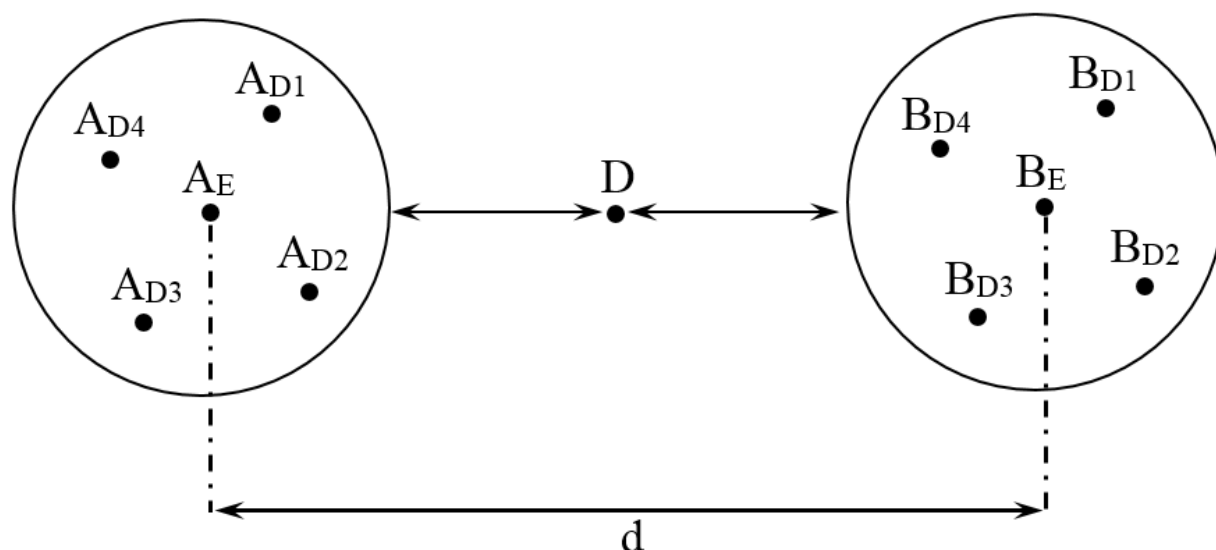


Рисунок 8. Сферы декодирования. A_E , B_E – разрешенные комбинации, A_{Di} , B_{Di} – запрещенные комбинации декодируемые в A_E или B_E , соответственно. D – Однозначно не декодируемая запрещенная комбинация.

На рисунке 8 схематично приведены две сферы декодирования, комбинации находящиеся в пределах сфер однозначно декодируются в A_E или B_E . При этом, на рисунке также показана комбинация D , которая находится на равном расстоянии от границ сфер комбинаций A_E и B_E , таким образом, однозначно декодировать такую комбинацию невозможно. Декодер, который осуществляет декодирования комбинаций только находящихся в пределах сфер декодирования называется неполным, в случае попытки декодирования комбинации D декодер формирует защитный отказ. Декодер, который осуществляет декодирование всех комбинаций называется полным, но при этом, комбинации по типу комбинации D показанной на рисунке 8 фактически декодируется случайным образом, такая комбинация фактически «присоединяется» к одной из сфер. Применение такого декодера целесообразно в том случае, если требуется хоть какая-то информация о принятом сообщении или например, в случае применения так называемых турбо-кодов, кода информация кодируется несколько раз, и ошибка может быть исправлена на следующем шаге декодирования.

Ранее нами был рассмотрен подход к декодированию при котором осуществлялось хранение всех разрешенных комбинаций, а декодирование заключалось в поиске комбинации наиболее близкой в смысле кодового расстояния к декодируемой. Существует подход позволяющий несколько формализовать, как построение сфер декодирования, так и сам процесс декодирования. Для этого формируется так называемая таблица стандартного расположения или таблица расположения по Слепяну. Такая таблица связывает разрешенные и соответствующие им запрещенные комбинации, иначе говоря, описывает сферы декодирования. Алгоритм построения таблицы основан на записи сферы декодирования нулевой комбинации – эта сфера будет содержать все комбинации веса меньше или равного t . То есть фактически эта содержит все возможные вектора исправляемых ошибок. А затем фактически осуществляется сдвиг системы координат. На практике все оказывается несколько проще. Пусть имеется код длины n , мощности M . Всего на этой длине формируется 2^n слов³. Первая строка содержит M столбцов, и состоит из разрешенных кодовых комбинаций, начиная с нулевой комбинации. Обозначим первую строку следующим образом: $\{0, C_2, C_3 \dots C_M\}$ (в данном случае комбинация C_1 обозначена нулем). После этого определим слово v_1 , которое лежит на расстояние 1 от нулевой комбинации, и заполним вторую строку таблицы, она примет вид:

$$\{v_1 + 0, v_1 + C_1, v_1 + C_2 \dots v_1 + C_M\}$$

На следующем шаге необходимо найти слово V_2 , которое будет ближайшим к нулевому, но не содержится в уже заполненной части таблицы. Аналогичным образом формируются остальные строки таблицы. Обобщим выражение описывающее содержимое очередной строки для i -го шага:

$$\{v_i + 0, v_i + C_1, v_i + C_2 \dots v_i + C_M\}$$

³ С формальной точки зрения слова лежат в расширении конечного поля, и правильно обозначить q^n , где q – размерность основного поля. Но в данном случае речь идет о двоичных кодах.

На каждом шаге необходимо следить за тем чтобы добавляемое слово не содержалось в заполненной части таблицы, в этом случае необходимо отметить соответствующую ячейку таблицы. Эта ячейка будет декодироваться в ближайшую комбинацию или будет формироваться защитный отказ, то есть в этом случае декодер фактически будет «угадывать» комбинацию, как было сказано ранее. После того, как все слова с весом равным t и меньше найдены и добавлены в первый столбец таблицы, следует либо прекратить процесс формирования таблицы либо отметить строку после которой декодер будет «угадывать» декодируемое слово. Это зависит от того реализуется полный или нет. Рассмотрим таблицу стандартного расположения для рассмотренного ранее кода длины $n = 3$, мощности $M = 2$ и кодового расстояния $d = 3$. Этот код исправляет одну ошибку, $t = 1$. Данный код имеет следующий набор разрешенных комбинаций: $\{000, 111\}$. Стандартное расположение этого кода показано в таблице 8.

Таблица 8. Пример таблицы стандартного расположения.

000	111
001	110
010	101
100	011

Рассмотрим формирование этой таблицы. Всего на $n = 3$ можно сформировать следующий набор из восьми комбинаций: $S = \{000, 001, 010, 011, 100, 101, 110, 111\}$. Этот набор содержит две разрешенные комбинации $\{000, 111\}$, остальные комбинации запрещены. Разрешенные комбинации были выписаны в первую строку таблицы. Теперь нужно найти комбинацию, которая будет в смысле кодового расстояния находится наиболее близко к нулевой комбинации, это комбинаций 001. Теперь путем суммирования разрешенных комбинаций формируется вторая строка таблицы, для этого просуммируем первую строку и комбинацию 001: $\{000 \oplus 001, 111 \oplus 001\} = \{001, 110\}$, этих

комбинаций в таблице пока нет. В результате набор S примет вид $S = \{010, 011, 100, 101\}$, из него были удалены разрешенные комбинации, а также комбинации из второй строки. Сформируем третью строку таблицы. Следующей ближайшей комбинацией к нулевой является комбинация 010, этой комбинации в таблице нет. Теперь сформируем третью строку $\{000 \oplus 010, 111 \oplus 010\} = \{010, 101\}$. Таких комбинаций в заполненной части таблицы нет. В результате чего, исходный набор S примет вид: $S = \{011, 100, 101\}$. Третья строка заполняется аналогичным образом, будет использована комбинация 100. При этом, набор S примет вид $S = \{011, 101\}$. Вес этих комбинаций превышает величину $t = 1$, следовательно процесс формирования таблицы окончен. Если внимательно посмотреть на первый столбец таблицы, то он фактически содержит вектора ошибок (нулевую комбинацию можно считать нулевым вектором ошибки). Рассмотрим пример декодирования комбинации 111. Эта комбинация находится в верхней строке таблицы, и следовательно является разрешенной, таким образом, никакие преобразования не требуются. Теперь рассмотрим декодирование комбинации 101. Найдем эту комбинацию в таблице. Данная комбинация расположена во втором столбце, но не в первой строке, следовательно она является запрещенной. Таким образом, данная комбинация декодируется в комбинацию находящуюся в первой строке найденного столбца, это комбинация 111.

Если рассмотреть построенный нами код, как алгебраическую подгруппу, то таблица стандартного расположения описывает деление этой подгруппы на так называемые смежные классы, а комбинации в первом столбце являются лидерами этих смежных классов. Такой способ декодирования еще называется декодированием по лидеру смежного класса.

Разумеется, ценность таблицы стандартного расположения в некотором смысле сомнительна, в особенности при росте мощности и корректирующей способности кода. Но в противовес этому утверждению можно сказать, что существует формальный алгоритм ее построения и возможно построить, как

полный, так и неполный декодер. В следующем разделе настоящего пособия будет рассмотрена возможность несколько сократить таблицу стандартного расположения, и избавиться от процедуры поиска декодируемой комбинации в таблице. Кроме этого, ранее нами был рассмотрен алгоритм кодирования, который основан на построении таблиц соответствия кодируемой (исходной) комбинации и комбинации принадлежащей коду. В последующем разделе также будет рассмотрен подход позволяющий не использовать такие таблицы соответствия.

3.5. Первая лабораторная работа

Теперь известна вся информация для выполнения первой лабораторной работы. В ходе первой лабораторной работы нужно разработать программу которая будет осуществлять следующие действия:

1. При заданной величине мощности и числе исправляемых ошибок формировать набор разрешенных и запрещенных комбинаций, в зависимости от варианта.
 - a. Необходимо определить длину кода n .
 - b. Реализовать алгоритм формирования разрешенных кодовых комбинаций.
 - c. Реализовать определение кодового расстояния для сформированного набора разрешенных комбинаций. В случае, если заданный набор не обладает требуемым расстоянием, то увеличить длину кода n или изменить алгоритм формирования комбинаций.
2. Разработать кодер, на вход которого подаются десятичные числа в диапазоне $[0; M-1]$, а на выходе кодера формируется разрешенная комбинация.
3. Разработать алгоритм внесения ошибки кратности от 1 до t в слово длины n , формируемое на выходе кодера.

Пусть $\{111\}$ – разрешенная комбинация, а $\{011\}$ – запрещенная комбинация. Переход от разрешенной к запрещенной комбинации происходит путем поразрядного суммирования по модулю два разрешенной комбинации с вектором ошибки. Вектор ошибки – это комбинация длины n , в которой единицы находятся в тех разрядах, где произойдет искажение. Для нашего примера вектор ошибки равен $\{100\}$. Вес вектора ошибки (число единиц) называется кратностью ошибки. Например, $\{100\}$ – вектор однократной ошибки, а $\{101\}$ – вектор двукратной ошибки.

4. Разработать декодер, на вход которого подается декодируемая комбинация, а на его выходе формируется декодированная комбинация (десятичная) или формируется сообщение об обнаруживаемой ошибке или отказ от декодирования, например, если осуществляется попытка декодирования комбинации не принадлежащей не одной из сфер декодирования.
5. Разработанная программа должна быть защищена от ввода ошибочных данных – превышения мощности кода, ввод строк вместо чисел и др.

4. Линейные коды

В предыдущем разделе нами была рассмотрена возможность построения корректирующего кода, кодер и декодер которого реализован в виде таблиц. В случае кодирования достаточно использовать кодируемую комбинацию в качестве адреса в такой таблице (использовать память с произвольным доступом, при аппаратной реализации или использовать массив, при программной). Для работы декодера будет необходимо сформировать таблицу по которой будет осуществляться декодирования или решать задачу поиска ближайшей разрешенной кодовой комбинации. Такие подходы подразумевают существенный рост затрат на память при росте мощности и корректирующей способности кода. Кроме этого, мы в явном виде не

обозначили один существенный недостаток построенных нами кодов. Слово принадлежащее такому коду вовсе не несет исходной информации, которую собственно и нужно передать, для этого требуются таблицы соответствия. Теперь перейдем к способу описания кода, который позволяет существенно минимизировать затраты подобного рода.

Существует класс так называемых линейных кодов, которые лишены таких недостатков. Линейным код называется в случае, если любая разрешенная комбинация, принадлежащая коду ζ , может быть выражена путем линейной комбинации других разрешенных комбинаций кода ζ . Разновидностью линейных кодов являются так называемые групповые систематические коды (ГСК). Данная группа кодов относится к группе блочных кодов, любое слово блочного кода имеет строго фиксированную длину, и в случае, если код является систематическим, то слово делится на две части – информационную и избыточную, структура сообщения приведена на рисунке 9.

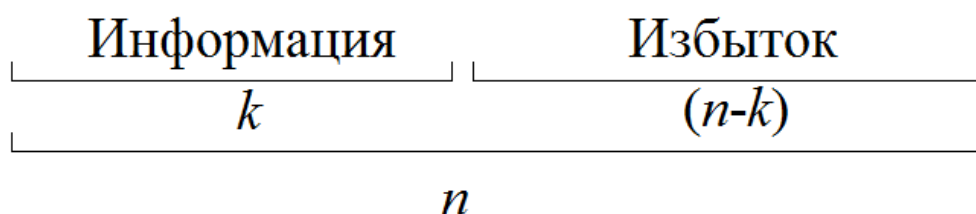


Рисунок 9. Структура слова ГСК.

Как видно из рисунка 9, слово принадлежащее коду состоит из n разрядов, которые в свою очередь, делятся на два блока. Первый блок, количество разрядов в котором равно k предназначен для кодируемой информации, при этом, $k < n$. Таким образом, слово ГСК в себе содержит исходную информацию, и таблица соответствия может не требоваться. Второй блок, количество разрядов которого равно $n-k$ предназначен для хранения контрольных (избыточных) разрядов. Значения избыточных разрядов формируется путем комбинации значений информационных разрядов.

Немного отвлечемся от структуры сообщения ГСК. Задача кодирования, как и было рассмотрено ранее является задачей определения соответствия между словами двух наборов. Первый набор – это кодируемые данные, а второй набор – это набор разрешенных слов некоторого кода ζ , который обладает требуемым кодовым расстоянием. В данном разделе далее для общности код будет называться просто линейным.

4.1. Матричное описание линейного кода.

Как было сказано ранее, операция кодирования – это фактически поиск соответствия между кодируемыми данными и разрешенными комбинациями кода. Выполнить такое отображение без применения таблиц соответствия возможно, если рассмотреть набор слов кода ζ в виде базиса. Базис можно задать при помощи матрицы, которая имеет следующий вид:

$$G = [I_{k \times k} Q_{k \times (n-k)}]_{k \times n}$$

Такая матрица задает базис, состоящий из векторов размерности n . В рамках рассматриваемой нами дисциплины такую матрицу будем называть матрицей порождающей код ζ или порождающей матрицей. Введем ограничения на размерность матрицы, а ограничения на ее содержимое введем немного позже. Количество столбцов матрицы G равно n – длине кода, а число строк равно k – количеству информационных разрядов в слове принадлежащему коду, порожденному матрицей G . Порождающая матрица является сжатым описанием кода – комбинируя ее строки можно получить любое разрешенное слово (свойство линейности кода). В случае, если матрица имеет единичную часть, как в случае, показанном выше, то матрица порождает так называемый систематический код. Систематический код – это код слово которого строго разделено на информационную и избыточную часть, как показано на рисунке 9. Матрица Q входящая в матрицу G приведенную выше называется приписной частью порождающей матрицы. Таким образом, операцию кодирования можно осуществить, выполнив разложение

кодируемого слова по базису заданному матрицей G . Разложение вектора f по базису можно выполнить при помощи матричного умножения:

$$F = fG$$

В результате чего будет сформировано разрешенное слово F принадлежащее коду порожденному матрицей G , а f – кодируемое слово. Количество разрядов слова F равно n , а слова f равно k . В двоичном случае данный код будет иметь возможность кодировать 2^k слов, иначе говоря, мощность кода $M = 2^k$. Таким образом, кодирование линейного кода сводится к выполнению матричного умножения и не требует применения таблиц. При этом, в двоичном случае, суммирование при выполнении матричного умножения осуществляется по модулю два. Полученное слово F передается по каналу связи, при этом, канал связи вносит искажения в передаваемое слово, этот процесс описывается уже рассмотренным нами ранее выражением:

$$\tilde{F} = F + V,$$

где $\tilde{F}$ – слово полученное из канала связи, которое в дальнейшем будет декодировано, V – вектор ошибки. Вектором ошибки будем считать слово, длина которого равна длине слова F , в котором единицы расположены на искаженных позициях. В процессе передачи вектор ошибки может быть нулевым, что соответствует отсутствию искажений. Число единиц в векторе ошибки (его вес) равен кратности возникшей ошибки или иначе говоря числу искаженных разрядов.

Теперь рассмотрим декодирование, в ходе которого необходимо определить закодированное сообщение и определить наличие ошибки. Декодирование также основано на рассмотрении слов принадлежащих коду в виде базиса. Если матрица G задает базис разрешенных кодовых слов, то при выполнении декодирования необходимо определить принадлежит ли декодируемое слово базису, заданному матрицей G . Разумеется, такую задачу можно решить перебором или при помощи таблицы стандартного

расположения, как было рассмотрено ранее. Однако, есть более эффективный способ, который также будет сведен к матричному умножению. Разрешенные слова кода ζ являются векторами в n – мерном пространстве, если слово является вектором, то для любого вектора можно получить ортогональный ему вектор. В случае, если два вектора ортогональны, то их скалярное произведение равно нулю. На основе этого свойства строится алгоритм декодирования линейного кода. Пусть F – вектор, который является разрешенным словом кода ζ , $F^\perp$ – ортогональный ему вектор, а $\tilde{F}$ – декодируемое слово. В случае, если $\tilde{F} = F$, то скалярное произведение $(\tilde{F} \times F^\perp)$ будет равно нулю, а в противном случае произведение будет ненулевым. Таким образом, заменив каждый разрешенный вектор ортогональным ему вектором, становится возможным определить принадлежность декодируемого вектора (слова) к набору разрешенных слов кода ζ . Набор векторов ортогональных векторам некоторого базиса называют ортогональным дополнением базиса. Матрица задающая ортогональное дополнение в двоичном случае строится на основе матрицы G следующим образом:

$$H = [Q^T \quad I]_{(n-k) \times n}$$

Матрица H называется проверочной матрицей кода. В случае, если матрица G не имеет единичной части, то для получения ортогонального дополнения, матрицу G необходимо привести к такому виду комбинируя ее строки. Порождающая матрица задает базис ортогональных векторов, таким образом, определить наличие ошибки возможно путем перемножения декодируемого слова и проверочной матрицы:

$$S = \tilde{F}H^T$$

Произведение S обычно называют синдромом, в случае, если синдром нулевой, то слово $\tilde{F}$ не содержит искажений. Теперь возникает задача определения местоположения искаженных разрядов в слове $\tilde{F}$, для этого необходимо определить вектор ошибки, после чего осуществить сложение $\tilde{F} +$

V , тем самым осуществив коррекцию. Для этого обратимся к рассмотренной нами ранее таблицы стандартного расположения, которая связывала вектора ошибок и разрешенные комбинации. Для того чтобы не обращаться к прошлым разделам, рассмотрим приведенное в таблице 9 описание стандартного расположения в общем виде.

Таблица 9. Общий вид таблицы стандартного расположения

	A_1	A_2	...	A_M
V_1	Y_{11}	Y_{12}	...	Y_{1M}
V_2	Y_{21}	Y_{22}	...	Y_{2M}
...	...	...	...	...
V_l	Y_{l1}	Y_{l2}	...	Y_{lM}

Первая строка таблицы содержит разрешенные комбинации $A_1 - A_M$, а первый столбец содержит возможные вектора ошибок $V_1 - V_M$. В выделенной части таблицы содержатся искаженные комбинации – смежные классы, значение которых формируется путем суммирования разрешенной комбинации и вектора (лидера смежного класса), например:

$$Y_{11} = A_1 + V_1$$

Так как мы рассматривает двоичные коды, то суммирование ведется по модулю два. Ранее нами было введено понятие синдрома ошибки, значение которого рассчитывается путем произведения декодируемой комбинации и порождающей матрицы кода. Теперь предположим, что существует взаимосвязь между синдромами ошибок и векторами $V_1 - V_M$, а именно каждому вектору ошибки (лидеру смежного класса) соответствует строго один синдром. На практике это приведет к тому, что произведение YH^T для любого Y из одного смежного класса будет иметь одинаковое значение. Это позволит существенно уменьшить объем таблицы стандартного расположения, и

хранить только значения векторов ошибок и их синдромов. Докажем это утверждение. Пусть Y и $\hat{Y}$ принадлежат одному смежному классу и равны:

$$\begin{aligned} Y &= V + A \\ \hat{Y} &= V + \hat{A} \end{aligned}$$

где V – вектор ошибки, A и $\hat{A}$ – пара разрешенных комбинаций, а так как вектор V в данных выражениях фиксирован, то Y и $\hat{Y}$ будут принадлежать одному смежному классу. Комбинации Y и $\hat{Y}$ фактически являются декодируемыми комбинациями. Теперь запишем выражения для расчета синдромов рассмотренных комбинаций:

$$\begin{aligned} S &= YH^T = (V + A)H^T = VH^T + AH^T \\ \hat{S} &= \hat{Y}H^T + (V + \hat{A})H^T = VH^T + \hat{A}H^T \end{aligned}$$

Ранее было установлено, что синдром при декодировании разрешенной комбинации равен нулю, отсюда:

$$\begin{aligned} AH^T &= 0 \\ \hat{A}H^T &= 0 \end{aligned}$$

Таким образом, выражения для расчета синдромов для Y и $\hat{Y}$ примут вид:

$$\begin{aligned} S &= VH^T \\ \hat{S} &= VH^T \end{aligned}$$

В результате чего, в случае, если имеются две искаженные разрешенные одним вектором комбинации, в расчете синдромов фактически будут участвовать только вектор ошибки, отсюда можно сделать вывод, что $S = \hat{S}$ и значения синдромов для комбинаций из одного смежного класса будут одинаковыми. Таким образом, таблица 9 изменится до вида представленного в таблице 10.

Таблица 10. Общий вид таблицы соответствия векторов и синдромов

Вектор ошибки	Синдром ошибки
V_1	S_1
V_2	S_2
...	...

Таблица 10 содержит в себе только векторы (лидеры смежных классов) и соответствующие им синдромы. Забегая вперед следует сказать, что синдромы и векторы в такой таблице должны быть уникальными, иначе коррекция будет выполнена неверно. Однако, если код обнаруживает s ошибок, то фрагмент таблицы соответствующий ошибкам кратности s будет иметь одинаковые синдромы.

Подведем итог. После того, как было установлено, что одному смежному классу (одному вектору ошибки) соответствует один синдром, то алгоритм декодирования линейного кода будет сведен к расчету синдрома, и поиску по таблице, аналогичной таблице 10 соответствующего ему вектора ошибки, с последующей ее коррекцией. Формирование таблицы соответствия векторов и синдромов в некотором роде аналогична построению таблицы стандартного расположения. Построение таблицы соответствия сводится к формированию всех допустимых векторов ошибок с последующим их декодированием и записью синдрома в таблицу. Общее количество векторов ошибок зависит от кратности исправляемой ошибки и определяется выражением:

$$N_s = \sum_{i=1}^t C_n^i,$$

где n – длина кода, t – число исправляемых ошибок, а C_n^i – число сочетаний из n по i .

Рассмотрим пример кода $(7,4,3)$, который имеет кодовое расстояние равное трем, и следовательно исправляет одну ошибку. Данный код порождается матрицей G которая имеет вид:

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

В результате рассмотренных ранее преобразований из порождающей матрицы сформируем проверочную матрицу H :

$$H = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Данный код обладает мощностью равной 16 сообщениям, и может закодировать комбинации десятичные значения которых находятся в диапазоне от нуля до 15. В двоичном виде это комбинации $\{0000, 0001, 0010 \dots 1111\}$. Теперь для возможности декодирования необходимо составить таблицу векторов и их синдромов, для этого нужно сформировать значения векторов. Нам заранее известно, что этот код обладает возможностью исправления одной ошибки, следовательно всего векторов будет семь, так как $C_7^1 = 7$. В таблице 11 приведены вектора ошибок для данного кода.

Таблица 11. Таблица векторов для кода $(7,4,3)$.

V_1	0 0 0 0 0 0 1
V_2	0 0 0 0 0 1 0
V_3	0 0 0 0 1 0 0
V_4	0 0 0 1 0 0 0
V_5	0 0 1 0 0 0 0
V_6	0 1 0 0 0 0 0
V_7	1 0 0 0 0 0 0

Таблица 11 содержит семь векторов ошибок кратности один. Теперь необходимо каждый вектор из таблицы 11 умножить на H^T , что фактически будет соответствовать декодированию искаженной нулевой комбинации. В таблице 12 приведено соответствие векторов и их синдромов для кода порожденного матрицей G приведенной выше. Разумеется, что суммирование при выполнении матричного умножения ведется по модулю два.

Таблица 12. Таблицы соответствия векторов и синдромов кода (7,4,3).

Вектор	Синдром
0 0 0 0 0 1	100
0 0 0 0 1 0	010
0 0 0 1 0 0	001
0 0 1 0 0 0	111
0 1 0 0 0 0	110
1 0 0 0 0 0	101
1 0 0 0 0 0	011

Теперь есть вся информация для работы с кодом, рассмотрим пример кодирования, декодирования с ошибкой и без. Напомним, что данный код обладает мощностью в 16 двоичных слов. В качестве примера будем кодировать сообщение $\varphi = 1101$. Кодирование ведется путем умножения слова φ на порождающую матрицу:

$$F = \varphi G = [1 \quad 1 \quad 0 \quad 1] \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix} = [1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0]$$

В результате сообщение $\varphi = 1101$ закодированное кодом порожденным матрицей G примет вид $F = 1101100$. В результате того, что используемая нами матрица записана в систематической форме, первые четыре разряда слова F равны слову φ . Теперь рассмотрим процесс передачи данных по каналу связи, в соответствии с рассмотренным ранее выражением $\tilde{F} = F \oplus V$, где $\tilde{F}$ – слово

полученное из канала связи, а V – вектор ошибки, который был добавлен к передаваемому слову F . Рассмотрим случай нулевого вектора ошибки, следовательно сообщение было передано без ошибки и $\tilde{F} = F$. Как было сказано ранее, декодирование сводится к расчету синдрома и поиску его вектора, расчет синдрома в свою очередь, выполняется путем умножения декодируемого слова на транспонированную проверочную матрицу:

$$S = \tilde{F}H^T = [1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0] \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = [0 \ 0 \ 0]$$

В результате умножения установлено, что синдром равен нулю, следовательно слово $\tilde{F}$ ошибок не содержит. Теперь внесем искажение, пусть вектор ошибки $V = 0100000$, что соответствует искажению следующего за старшим разрядом, в результате декодируемое сообщение примет вид:

$$\begin{aligned} \tilde{F} = F + V &= [1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0] + [0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0] \\ &= [1 \quad \mathbf{0} \quad 0 \quad 1 \quad 1 \quad 0 \quad 0] \end{aligned}$$

Искаженный разряд отмечен полужирным шрифтом. Теперь рассчитаем синдром для данного сообщения:

$$S = \tilde{F}H^T = [1 \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0] \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = [1 \ 0 \ 1]$$

В результате был получен синдром равный 101, теперь обратимся к соответствию векторов и их синдромов, приведенных в таблице 12. Исходя из данных приведенных в таблице, этому синдрому соответствует вектор $V_t = 0100000$. Теперь осуществим коррекцию ошибки, которая, как нетрудно

догадаться заключается в суммировании полученного из таблицы вектора и декодируемого сообщения:

$$F_{corr} = F + V_t = [1 \ 0 \ 0 \ 1 \ 1 \ 0 \ 0] + [0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0] \\ = [1 \ 1 \ 0 \ 1 \ 1 \ 0 \ 0]$$

4.2. Описание линейного кода при помощи систем уравнений.

Кроме описания кодирования и декодирования при помощи матричных операций существует возможность описания этих процессов при помощи систем уравнений. Используется две системы уравнений – системы уравнений кодирования и системы уравнений проверок. Обе эти системы строятся на базе проверочной матрицы H , при этом, матрица должна быть в систематической форме – содержать единичную часть. Сравним структуру сообщения систематического кода и структуру его порождающей матрицы:

$$G = \begin{bmatrix} a_1 & a_2 & \dots & a_k & a_{(k+1)} & \dots & a_n \\ & & & I & & & Q \\ & & & & & & \end{bmatrix} \begin{matrix} k \\ n-k \\ n \end{matrix}$$

Информация
Избыток

k
 $n-k$

n

Рисунок 10. Сравнение структуры порождающей матрицы и сообщения.

Единичная часть матрицы G соответствует информационной части сообщения (разряды $a_1 - a_k$), а приписная часть контрольной (избыточной, разряды $a_{k+1} - a_n$), таким образом, при формировании проверочной матрицы подобная структура сохранится – транспонированная приписная часть будет отвечать за информационную часть сообщения, а единичная за избыточную:

$$H = \begin{bmatrix} a_1 & a_2 & \dots & a_k & a_{(k+1)} & \dots & a_n \\ & & Q^T & & & & I \end{bmatrix}_{n-k}$$

Информация
Избыток

k
 $n-k$

n

Рисунок 11. Сравнение структуры проверочной матрицы и сообщения.

Как видно из рисунка 11, в каждой строке проверочной матрицы «участвует» только один проверочный разряд. Вспомним, что при помощи выражения $\tilde{F}H^T = 0$ осуществлялась проверка наличия ошибки в декодируемом слове. Данное выражение является системой линейных уравнений, которая называется системой уравнений проверок. В этом случае процесс декодирования сводится к поразрядному умножению сообщения на каждую строку матрицы H , то есть фактически этот процесс можно переписать следующим образом:

$$\left\{ \begin{array}{l} \xleftarrow{Q^T} \quad \quad \quad \xleftarrow{I} \\ h_{11}a_1 \oplus h_{12}a_2 \oplus \dots \oplus h_{1k}a_k \oplus a_{k+1}1 \oplus a_{k+2}0 \oplus a_n 0 = 0 \\ \vdots \\ h_{(n-k)a_1} \oplus h_{(n-k)a_2} \oplus \dots \oplus h_{(n-k)a_k} \oplus a_{k+1}0 \oplus a_{k+2}0 \oplus a_n 1 = 0 \end{array} \right.$$

Каждое уравнение данной системы состоит из двух частей – первая основана на транспонированной приписной части, а вторая на единичной диагональной, а символом $h_{ij}, i = \overrightarrow{1, k}, j = \overrightarrow{1, (n-k)}$ обозначены соответствующие элементы матрицы H . Таким образом, подставив разряды $a_1 - a_n$ декодируемого сообщения в систему уравнений проверок, получим результат аналогичный умножению $\tilde{F}H^T$, правая часть системы уравнений также будет являться синдромом ошибки. Соответственно таблица соответствия векторов и

синдромов строится путем последовательной подстановки векторов в систему уравнений проверок и записью ее правой части.

Теперь рассмотрим способ кодирования линейного кода с использованием такой системы уравнений. Как было сказано ранее, каждая строка проверочной матрицы «содержит» в себе ровно один проверочный разряд, так и в случае системы уравнений проверок, каждое уравнение содержит ровно один проверочный разряд. Разрешив каждое уравнение относительно проверочного разряда, получим систему уравнений кодирования, которая примет вид:

$$\begin{cases} a_{k+1} = h_{11}a_1 \oplus h_{12}a_2 \oplus \dots \oplus h_{1k}a_k \\ \dots \\ a_n = h_{(n-k)}a_1 \oplus h_{(n-k)}a_2 \oplus \dots \oplus h_{(n-k)}a_k \end{cases}$$

Левая часть системы уравнений проверок определяет значения контрольных разрядов слова принадлежащего коду, значение каждого из контрольных разрядов определяется комбинацией разрядов кодируемого слова. Иначе говоря, при выполнении операции кодирования известна только правая часть системы уравнений проверок и матрица H .

Перейдем к примеру кодирования и декодирования сообщений при помощи систем уравнений. Для разнообразия будем использовать другой код – код $(5,2,3)$, который порожден матрицей G вида:

$$G = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

Первые два столбца являются единичной матрицей, а последующие три задают присписную часть матрицы. На основе порождающей матрицы сформируем проверочную матрицу:

$$H = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

На основе матрицы H сформируем систему уравнений проверок:

$$\begin{cases} a_1 \oplus a_2 \oplus a_3 = 0 \\ a_1 \oplus a_4 = 0 \\ a_1 \oplus a_2 \oplus a_5 = 0 \end{cases}$$

Путем разрешения каждого уравнения относительно контрольного разряда получим систему уравнений кодирования:

$$\begin{cases} a_3 = a_1 \oplus a_2 \\ a_4 = a_1 \\ a_5 = a_1 \oplus a_2 \end{cases}$$

Данный код имеет два двоичных информационных разряда, таким образом, его мощность $M = 2^k = 2^2 = 4$. Множество кодируемых кодом ζ слов имеет вид: {00, 01, 10, 11}. Теперь сформируем возможные вектора ошибок, код ζ имеет кодовое расстояние равно трем, в результате чего, код ζ обладает возможностью коррекции однократных ошибок и их общее число будет равно $C_5^1 = 5$. Данный код будет иметь следующие допустимые вектора ошибок:

$$\begin{array}{l} V_1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 1 \\ V_2 \quad 0 \quad 0 \quad 0 \quad 1 \quad 0 \\ V_3 \quad 0 \quad 0 \quad 1 \quad 0 \quad 0 \\ V_4 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \\ V_5 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \end{array}$$

Теперь сформируем соответствующие синдромы ошибок, для чего поочередно «подставим» каждый вектор в систему уравнений проверок, в качестве примера рассмотрим вектор V_3 , при этом, разряды вектора ошибки нумеруются аналогично разрядам сообщения:

$$V_3 = \begin{matrix} a_1 & a_2 & a_3 & a_4 & a_5 \\ 0 & 0 & 1 & 0 & 0 \end{matrix} \rightarrow \begin{cases} a_1 \oplus a_2 \oplus a_3 = 0 \oplus 0 \oplus 1 = 1 \\ a_1 \oplus a_4 = 0 \oplus 0 = 0 \\ a_1 \oplus a_2 \oplus a_5 = 0 \oplus 0 \oplus 0 = 0 \end{cases}$$

Таким образом, синдром вектора $V_3 = 100$, аналогичным образом формируются остальные синдромы, в результате чего формируется таблица векторов и их синдромов:

V_1	0	0	0	0	1	S_1	1	1	1
V_2	0	0	0	1	0	S_2	1	0	1
V_3	0	0	1	0	0	S_3	1	0	0
V_4	0	1	0	0	0	S_4	0	1	0
V_5	1	0	0	0	0	S_5	0	0	1

Разумеется, аналогичную операцию расчета значений синдромов можно осуществить путем умножения вектора на матрицу H^T . Данные процедуры являются общим алгоритмом расчета значений синдромов. Внимательный читатель заметит, что значения синдромов в рассмотренном нами примере равны столбцам порождающей матрицы. В случае однократных ошибок синдромы всегда будут иметь такие значения.

После того, как все необходимые наборы данных определены, перейдем к рассмотрению работы кода, рассмотрим операцию кодирования. Как было сказано ранее, существует два способа кодирования – матричным умножением и подстановкой в систему уравнений кодирования. Рассмотрим кодирование слова $\varphi=10$ при помощи системы уравнений. Таким образом, на первом этапе кодирования известна только часть слова принадлежащего коду, структура которого приведена на рисунке 12:

$$\begin{array}{c}
 1 \ 0 \ \boxed{} \ ? \\
 a_1 \ a_2 \ a_3 \ a_4 \ a_5 \\
 \boxed{} \ \boxed{} \\
 \boxed{k} \ \boxed{n-k} \\
 \phantom{\boxed{}} \ n
 \end{array}$$

Рисунок 12. Слово принадлежащее коду до расчета контрольных разрядов.

Как видно из рисунка 12, операция кодирования начинается с подстановки в слово информационных разрядов, на рисунке это разряды a_1 и

a_2 , а разряды $a_3 - a_5$ пока неизвестны. После чего значения разрядов a_1 и a_2 «подставляются в систему уравнений кодирования:

$$\begin{cases} a_3 = a_1 \oplus a_2 = 1 \oplus 0 = 1 \\ a_4 = a_1 = 1 \\ a_5 = a_1 \oplus a_2 = 1 \oplus 0 = 1 \end{cases}$$

Таким образом, сообщение примет вид:

$$\begin{array}{ccccc} 1 & 0 & 1 & 1 & 1 \\ a_1 & a_2 & a_3 & a_4 & a_5 \\ \hline \underbrace{\hspace{1.5cm}}_k & \underbrace{\hspace{1.5cm}}_{n-k} & & & \\ & n & & & \end{array}$$

Теперь перейдем к осуществлению операции декодирования, в случае отсутствия ошибки, для этого декодируемое сообщение $\tilde{F} = 10111$ подставим в систему уравнений проверок:

$$\begin{array}{ccccc} a_1 & a_2 & a_3 & a_4 & a_5 \\ \tilde{F} = [1 & 0 & 1 & 1 & 1] \end{array} \rightarrow \begin{cases} a_1 \oplus a_2 \oplus a_3 = 1 \oplus 0 \oplus 1 = 0 \\ a_1 \oplus a_4 = 1 \oplus 1 = 0 \\ a_1 \oplus a_2 \oplus a_5 = 1 \oplus 0 \oplus 1 = 0 \end{cases}$$

После расчета значений правых частей уравнений становится очевидным отсутствие ошибки – синдром равен нулю. Теперь рассмотрим декодирование в случае ошибки, пусть вносимый каналом связи вектор ошибки будет равен $V = 00100$, тогда сообщение $\tilde{F}$ будет равно $\tilde{F} = F + V = 10011$. Подставим это сообщение в систему уравнений проверок:

$$\begin{array}{ccccc} a_1 & a_2 & a_3 & a_4 & a_5 \\ \tilde{F} = [1 & 0 & 0 & 1 & 1] \end{array} \rightarrow \begin{cases} a_1 \oplus a_2 \oplus a_3 = 1 \oplus 0 \oplus 0 = 1 \\ a_1 \oplus a_4 = 1 \oplus 1 = 0 \\ a_1 \oplus a_2 \oplus a_5 = 1 \oplus 0 \oplus 1 = 0 \end{cases}$$

В результате расчета был получен синдром $S = 100$, теперь при помощи таблицы соответствия векторов и их синдромов, по найденному синдрому определим его вектор, который равен $V_3 = 00100$, после чего становится возможным выполнить коррекцию:

$$F = \tilde{F} + V_3 = [1 \ 0 \ 0 \ 1 \ 1] + [0 \ 0 \ 1 \ 0 \ 0] = [1 \ 0 \ 1 \ 1 \ 1]$$

4.3. Декодирование по лидеру смежного класса.

Ранее нами был рассмотрен еще один способ декодирования линейного кода, который назывался декодированием по лидеру смежного класса. Такой способ основан на применении так называемой таблицы стандартного расположения, которая связывает разрешенные, запрещенные комбинации, а также вектора ошибок. Множество разрешенных комбинаций определяет так называемый основной класс, а каждый из векторов ошибок формирует смежный к нему класс. Получим таблицу стандартного расположения для рассмотренного ранее кода (5,2,3):

Таблица 13. Таблица стандартного расположения кода (5,2,3)

0 0 0 0 0	1 0 1 1 1	0 1 1 0 1	1 1 0 1 0
0 0 0 0 1	1 0 1 1 0	0 1 1 0 0	1 1 0 1 1
0 0 0 1 0	1 0 1 0 1	0 1 1 1 1	1 1 0 0 0
0 0 1 0 0	1 0 0 1 1	0 1 0 0 1	1 1 1 1 0
0 1 0 0 0	1 1 1 1 1	0 0 1 0 1	1 0 0 1 0
1 0 0 0 0	0 0 1 1 1	1 1 1 0 1	0 1 0 1 0

Алгоритм формирования таблицы 13 был рассмотрен ранее, на нем останавливаться не будем. Лишь кратко напомним о структуре таблицы. Первая строка таблицы содержит все разрешенные слова кода, а первый

столбец все вектора ошибок. Остальная часть таблицы – это соответствующая сумма векторов ошибок и разрешенных комбинаций. Теперь рассмотрим декодирование следующих двух слов:

$$\tilde{F}_1 = F + V_0 = [1 \ 0 \ 1 \ 1 \ 1] + [0 \ 0 \ 0 \ 0 \ 0] = [1 \ 0 \ 1 \ 1 \ 1]$$

$$\tilde{F}_2 = F + V_3 = [1 \ 0 \ 1 \ 1 \ 1] + [0 \ 0 \ 1 \ 0 \ 0] = [1 \ 0 \ 0 \ 1 \ 1]$$

Слово $\tilde{F}_1$ не содержится в смежных классах, следовательно в нем нет ошибки, а слово $\tilde{F}_2$ содержится в смежном классе (второй столбец четвертой строки), лидер которого равен $[0 \ 0 \ 1 \ 0 \ 0]$, таким образом, исходное слово можно получить путем суммирования лидера и декодируемого слова либо получить из первой строки соответствующего столбца, таким образом, скорректированное слово равно $[1 \ 0 \ 1 \ 1 \ 1]$.